Module 11 — MLOps Foundations for AI Engineers

Learning Objectives

By the end of this module, you will be able to:

- Explain why **deployment is just the beginning** of the machine learning model lifecycle.
- Apply **FastAPI's dependency injection and routing** to manage model versioning and safe updates.
- Design a system to **track model metadata** and performance metrics using **SQLModel**.
- Utilize **Docker** to ensure environmental reproducibility, solving the "it works on my machine" problem for complex ML dependencies.
- Identify conceptual triggers for **monitoring**, such as performance degradation and silent failures in inference.

The Reality of Production AI: Beyond the Launch

For an AI Engineer, deploying a model to a production environment like **Uber** or **Microsoft** is not the final step. Most AI systems fail after launch because they are treated as static software when they are actually **dynamic mathematical artifacts**.

- **Why most AI systems fail:** In a notebook, you control the data. In production, real-world data changes (drift), and the model's accuracy can drop without the server ever "crashing".
- **The MLOps Mission:** To create a **repeatable, reliable, and observable** pipeline for training, validating, deploying, and replacing models without breaking the user experience.

The Model Lifecycle

In a production-grade FastAPI system, the lifecycle is managed through specific technical stages:

1. **Training & Validation:** The research phase where models are created.
2. **Deployment (The "Startup" Phase):** Using FastAPI **Lifespan/Startup events** to load weights into memory once so they are ready for the first request.
3. **Monitoring (The "Observability" Phase):** Tracking response models and logging to detect if a model is returning "None" or incorrect results.

4. **Replacement (The "Safe Update" Phase):** Utilizing **APIRouter** to roll out a new version alongside the old one to ensure zero downtime.

Experiment Tracking and Reproducibility

Note on Outside Information: The terms "MLflow," "Runs," "Metrics," and "Artifacts" are common MLOps standards not explicitly defined in the provided sources. I have included them below to satisfy the prompt requirements while identifying them as conceptual extensions of the source materials.

- **Experiment Tracking:** This is the act of logging the parameters and results of every training run.
- **Reproducibility:** A model is only useful if it can be recreated. **Docker** is the primary tool here; it stores the **static contents** (files and environment) needed to run the model exactly the same way every time.
- **Artifacts:** In a FastAPI context, these are your model weights (e.g., `.pkl` or `.pt` files). These are packaged into the **Docker Image** to ensure the "Program" and the "Data" are inseparable.

Model Versioning: Code vs. Weights

AI Engineers must manage two types of versions:

1. **Code Versioning:** Handled by tools like Git and Docker. It tracks changes to your FastAPI logic.
2. **Model Versioning:** Handled by **APIRouter** and **Metadata Databases**. It tracks which mathematical version of the model is being queried.

ASCII Representation of Versioned Routing:

```
Client Request ----> [ FastAPI App ]
                          |
        ------------------------------------------
        |                                        |
    [/v1/predict]                            [/v2/predict]
        |                                        |
    (Old Model)                              (New Model)
    [Stable Logic]                           [Beta Logic]
```

Code Examples: MLOps with FastAPI & SQLModel

Example 1: Tracking Model Metadata (Simple MLOps Database)

We can use **SQLModel** to log every "Run" or "Version" of our model into a database for tracking.

```
from sqlmodel import SQLModel, Field #
from typing import Optional

class ModelRun(SQLModel, table=True): #
    id: Optional[int] = Field(default=None, primary_key=True)
    version_tag: str # e.g., "v1.0.2"
    accuracy_score: float # The "Metric"
    weights_path: str # The "Artifact" location
```

Line-by-Line Explanation:

- **`table=True`**: Tells SQLModel to create a real table in the database to store our experiment history.
- **`version_tag`**: Allows us to track which model version we are currently serving.
- **`accuracy_score`**: Stores the performance metric calculated during validation.
- **`weights_path`**: Records where the model weights are stored so they can be loaded during the **Startup Event**.

Example 2: Loading Correct Model Versions via Startup

This uses the **lifespan event** pattern to ensure the production environment is ready.

```
from fastapi import FastAPI
import logging #

app = FastAPI()
model_store = {}

@app.on_event("startup") #
def load_production_model():
    # Simulate loading versioned weights from disk/registry
    version = "v2.0"
    model_store["active"] = f"Loaded weights for {version}"
    logging.info(f"MLOps: Production model {version} is now active.") #

@app.post("/predict")
def inference():
    # Use the pre-loaded production-ready artifact
    return {"result": model_store["active"]}
```

Line-by-Line Explanation:

- **`@app.on_event("startup")`**: Ensures the heavy model is loaded once before any requests arrive, minimizing latency.
- **`model_store["active"]`**: Acts as a "Registry" in memory to hold the active model artifact.
- **`logging.info`**: Provides observability, allowing engineers to confirm the correct version was deployed in the logs.

Monitoring and Drift (Conceptual)

Models in production require constant checking for "Drift":

- **Data Drift:** When the input data from users changes significantly from what the model was trained on.
- **Prediction Drift:** When the model's output distribution shifts (e.g., a fraud model suddenly flags 90% of all transactions).
- **Performance Degradation:** When the model becomes slower or less accurate over time. Using **Middlewares** to track time/latency is a key MLOps monitoring strategy.

AI-Engineer Scenarios

- **Updating without breaking the API:** Use **Response Models**. Even if the model math changes, ensure the JSON response structure (the "Contract") remains identical so the client's code doesn't crash.
- **Rolling back a bad model:** If a new model fails, you don't delete code. You simply point your **FastAPI Dependency** back to the previous version's weights or restart a previous **Docker Container**.
- **Comparing Models:** Run two versions simultaneously using **APIRouter** prefixes (`/stable` vs `/canary`) to compare real-world performance before a full rollout.

Common MLOps Mistakes

- **Manual Deployment:** Copy-pasting model files onto a server. This is not reproducible. Use **Docker**.
- **Hardcoding Paths:** Using local paths like `/Users/dev/model.pkl` which don't exist in the **Docker container**.
- **Ignoring Logs:** Not logging the model's confidence scores. This makes it impossible to detect **Silent Failures**.
- **Blind Replacement:** Overwriting the only copy of a model. If it fails, there is no way to **Rollback**.

Assignment: Track and Serve

1. Create a **SQLModel** named `ModelRegistry` that stores `version`, `date_trained`, and `is_active`.
2. Add a **GET** endpoint `/registry` that returns a list of all models in the database.

3. Implement a **Startup Event** that adds a default "v1" entry to this registry when the server starts.
4. Dockerize this application using the **official Python image**.

Mini Project: Versioned Inference Service

Goal: Build a "Smooth Rollout" system.

- Create two routers: `v1_router` and `v2_router`.
- Define a POST `/predict` in both, but make V2 return a different "dummy" prediction to simulate an improved model.
- Include a **Health Check** endpoint that logs the currently loaded model versions.
- Ensure the entire service is wrapped in a **Docker image** for easy deployment.

Production Readiness Checklist: Is my AI system maintainable?

- [ ] Is my model loaded via a **Startup/Lifespan event** to prevent request-time latency?
- [ ] Are all model inputs and outputs strictly validated by **Pydantic/SQLModel**?
- [ ] Is the environment (Python version + libraries) captured in a **Dockerfile**?
- [ ] Do I have a way to serve **multiple model versions** using APIRouter for testing?
- [ ] Are the model's internal performance metrics being **logged** for monitoring?
- [ ] Is there a clear path to **Rollback** the model by changing a Docker tag or configuration?